TRAVAUX PRATIQUES

Bases de Données & Modélisation des SI

TP 10 : Utiliser CASE COALESCE et NULLIF

Objectif : construire des expressions conditionnelles dans une requête SELECT, catégoriser les commandes et adapter les valeurs affichées dans AdventureWorks sous PostgreSQL.

Prérequis : SELECT, FROM, alias AS, comparaisons, BETWEEN et tests NULL. Ce TP fourni des explications, des manipulations progressives et des exercices d'application.

I- Comprendre une expression conditionnelle

Une expression conditionnelle choisit une valeur à restituer selon un test. Elle peut apparaître parmi les colonnes de SELECT et recevoir un alias. Elle transforme le résultat affiché sans modifier les données stockées. Les requêtes de ce TP sont uniquement des consultations.

CASE permet d'exprimer plusieurs cas. COALESCE choisit la première valeur non NULL parmi ses arguments. NULLIF compare deux expressions et peut produire NULL. Les noms déformés dans la transcription correspondent à ces trois expressions SQL.

II- Décomposer CASE WHEN THEN ELSE END

```
SELECT productid, name, color,
       CASE
           WHEN color IS NULL THEN 'No color'
           ELSE color
       END AS product_color
FROM production.product;
```

CASE ouvre l'expression. WHEN introduit une condition. THEN précède la valeur renvoyée lorsque cette condition est vraie. ELSE fournit le résultat si aucune condition n'est vraie. END termine l'expression ; AS product_color nomme la colonne calculée.

Pour une couleur absente, product_color affiche No color. Pour une couleur renseignée, ELSE conserve la valeur. La colonne color est affichée à côté pour comparer la donnée initiale et le résultat. Pensez à la virgule entre color et CASE : le résultat du CASE forme une nouvelle colonne.

Les conditions sont examinées dans l'ordre écrit et le premier WHEN vrai détermine le résultat. Plusieurs WHEN peuvent se succéder. Si ELSE est absent et qu'aucune condition n'est vraie, le résultat est NULL. Les valeurs renvoyées par les différentes branches doivent avoir des types compatibles ; les catégories utilisées ici sont toutes textuelles.

III- Associer un numéro de jour à un nom

Le texte propose une colonne day_of_week contenant un code de 1 à 7 : 1 pour lundi, 2 pour mardi, jusqu'à 7 pour dimanche. Cette convention décrit une donnée déjà présente ; elle ne demande pas de calculer le jour à partir d'une date.

```
-- Exemple conceptuel
SELECT DayOfWeek,
       CASE DayOfWeek
           WHEN 1 THEN 'Lundi'
           WHEN 2 THEN 'Mardi'
           WHEN 3 THEN 'Mercredi'
           WHEN 4 THEN 'Jeudi'
           WHEN 5 THEN 'Vendredi'
           WHEN 6 THEN 'Samedi'
           WHEN 7 THEN 'Dimanche'
       END AS DayName
FROM YourTable;
```

YourTable est un nom illustratif : ce n'est pas une table supposée exister dans AdventureWorks. Dans cet exemple, DayOfWeek contient déjà le code du jour. La forme CASE DayOfWeek compare cette valeur aux nombres placés après WHEN : WHEN 2 signifie ici « si DayOfWeek vaut 2 ». Contrairement à CASE WHEN condition, on ne répète pas une comparaison complète à chaque branche.

CASE — Associer un numéro au nom du jour

Données initiales

DayOfWeek
1
2
3
4
5
6
7

Requête SQL

```
SELECT DayOfWeek,
CASE DayOfWeek
WHEN 1 THEN 'Lundi'
WHEN 2 THEN 'Mardi'
WHEN 3 THEN 'Mercredi'
WHEN 4 THEN 'Jeudi'
WHEN 5 THEN 'Vendredi'
WHEN 6 THEN 'Samedi'
WHEN 7 THEN 'Dimanche'
END AS DayName
FROM YourTable;
```

Résultat

DayOfWeek	DayName
1	Lundi
2	Mardi
3	Mercredi
4	Jeudi
5	Vendredi
6	Samedi
7	Dimanche

Figure 1 — Correspondance entre le numéro et le nom du jour

Lecture de la capture : le tableau de gauche contient les codes 1 à 7 ; la requête au centre associe chaque code à un nom ; le tableau de droite conserve DayOfWeek et ajoute DayName. Ainsi, 1 devient Lundi, 4 devient Jeudi et 7 devient Dimanche. Si un code différent était présent, il produirait NULL en l'absence de ELSE. Les données de départ ne sont pas modifiées.

IV- Remplacer une valeur absente avec COALESCE

```
SELECT productid, name, color,
       COALESCE(color, 'No color') AS product_color
FROM production.product;
```

COALESCE lit ses arguments de gauche à droite et restitue le premier qui n'est pas NULL. Ici, il conserve color si cette valeur est renseignée ; sinon, il affiche No color. Ce résultat correspond au CASE de la section II.

Le critère porte sur l'absence de valeur. Une chaîne vide, le texte Black ou le nombre zéro ne sont pas NULL. COALESCE ne remplace donc pas automatiquement ces valeurs. Il peut recevoir plus de deux arguments ; si tous sont NULL, le résultat est NULL. Ses arguments doivent pouvoir produire un même type de résultat.

```
SELECT COALESCE(NULL, 'Red', 'No color') AS exemple;
```

Cet exemple retourne Red : après le premier argument NULL, la première valeur renseignée est Red.

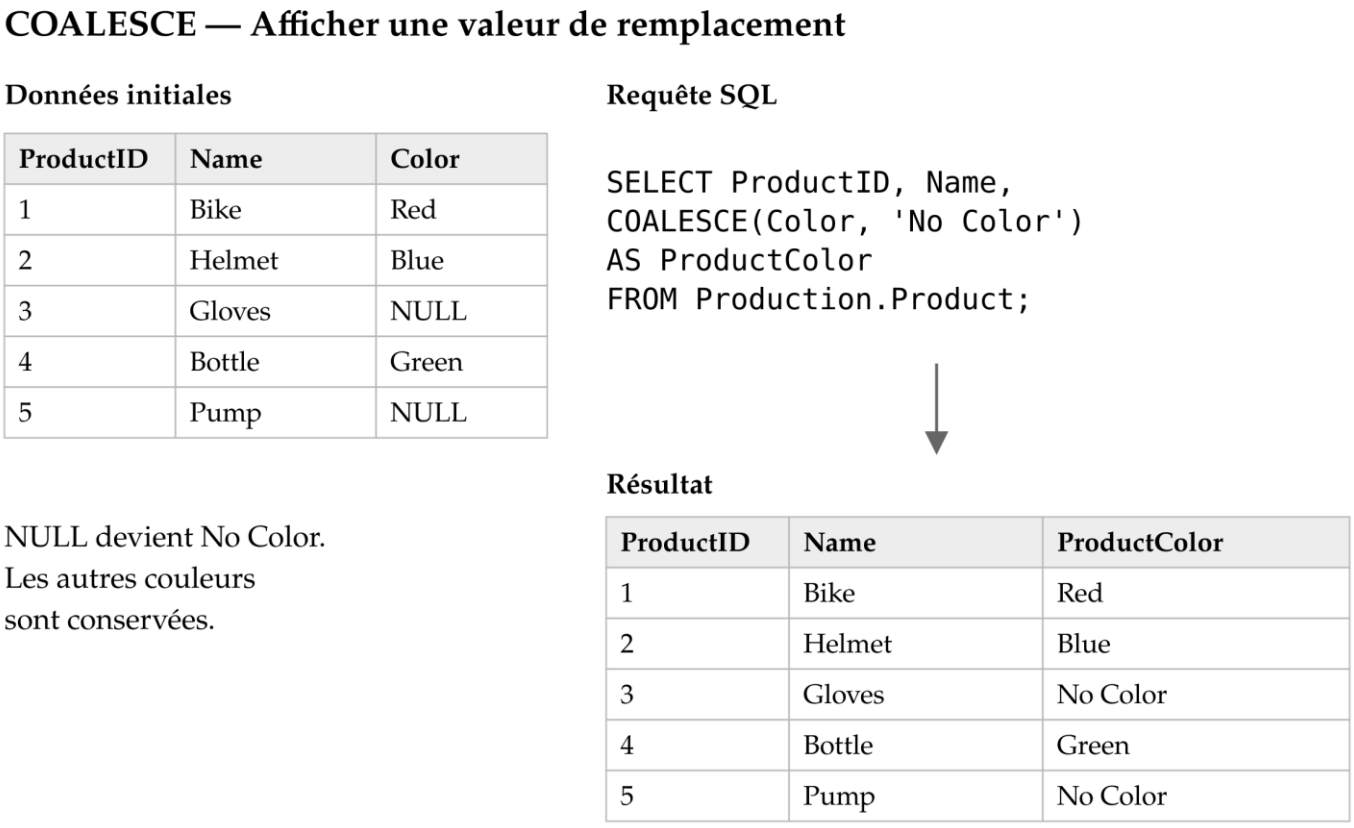


Figure 2 — Remplacement des couleurs absentes avec COALESCE

Lecture de la capture : Bike conserve Red, Helmet conserve Blue et Bottle conserve Green. Pour Gloves et Pump, la couleur NULL devient le libellé No Color dans ProductColor. Les cinq produits

restent présents : le traitement change une valeur affichée, sans filtrer les lignes. La majuscule de Color dans ce libellé reproduit la capture ; le TP utilise aussi No color comme texte de remplacement.

V- Comparer deux valeurs avec NULLIF

NULLIF(expression1, expression2) retourne NULL lorsque les deux expressions sont égales. Sinon, il retourne la première expression. Les arguments doivent être comparables. Cette expression ne prend pas un texte de remplacement en troisième argument.

```
SELECT NULLIF(10, 10) AS valeurs_egales,
       NULLIF(10, 0) AS valeurs_differentes,
       NULLIF(0, 0) AS zero_remplace;
```

Les trois résultats sont respectivement NULL, 10 et NULL. L'ordre des arguments compte : NULLIF(10, 0) conserve 10, tandis que NULLIF(0, 10) conserve 0. On choisit donc soigneusement la première expression à conserver en l'absence d'égalité.

VI- Protéger une division par zéro

```
SELECT name, listprice, standardcost,
       listprice / NULLIF(standardcost, 0) AS markup
FROM production.product;
```

Lorsque standardcost vaut zéro, NULLIF le remplace par NULL dans le dénominateur. Le ratio renvoyé est alors NULL, ce qui évite la division par zéro. Lorsque le coût est différent de zéro, le calcul utilise sa valeur initiale. Si le coût est déjà NULL, le ratio reste NULL.

Exemples : avec un prix de 120 et un coût de 60, le ratio vaut 2. Avec un prix de 120 et un coût de 0, il vaut NULL. NULL ne signifie pas ici un ratio nul : le ratio n'est pas calculable avec les informations retenues.

L'alias markup reprend celui du texte. L'expression calcule exactement prix catalogue divisé par coût standard. Elle ne représente pas directement un bénéfice ni un pourcentage de marge. On peut également écrire des additions, soustractions et multiplications entre colonnes dans SELECT.

NULLIF — Protéger une division par zéro

Données initiales

ProductID	Name	ListPrice	StandardCost
1	A	1000	500
2	B	2000	0
3	C	3000	1500
4	D	4000	2000
5	E	5000	0

Si le coût vaut 0,
NULLIF renvoie NULL.

Le ratio devient NULL :
aucune division par zéro.

Requête SQL

```
SELECT ProductID, Name,
ListPrice /
NULLIF(StandardCost, 0)
AS Markup
FROM Production.Product;
```

Résultat

ProductID	Name	Markup
1	A	2
2	B	NULL
3	C	2
4	D	2
5	E	NULL

Figure 3 — Division protégée par NULLIF sur le coût standard

Lecture de la capture : A donne 1000 / 500 = 2, C donne 3000 / 1500 = 2 et D donne 4000 / 2000 = 2. Pour B et E, StandardCost vaut zéro : NULLIF(StandardCost, 0) produit NULL, puis le ratio Markup est NULL. Le calcul n'attribue donc pas un ratio égal à zéro à ces produits.

Les produits et les effectifs visibles dans les figures 1 à 3 sont des exemples pédagogiques fournis avec le texte. Ils ne constituent pas des résultats à retrouver obligatoirement dans votre sauvegarde AdventureWorks. Les requêtes pratiques conservent les noms de colonnes PostgreSQL de votre base.

PARTIE PRATIQUE DANS ADVENTUREWORKS

VII- Préparer les tables et le script

1. Ouvrez Query Tool sur AdventureWorks et enregistrez TP10_Nom_Prenom.sql. Vérifiez les noms des tables et des colonnes avant d'exécuter les exemples.

2. Repérez sales.salesorderheader avec salesorderid et totaldue ; production.product avec productid, name, color, listprice et standardcost ; puis humanresources.employee avec businessentityid et hiredate.

Les exemples utilisent des identifiants en minuscules. Si votre sauvegarde emploie des noms délimités avec des majuscules, reprenez leur orthographe exacte entre guillemets doubles. Les chaînes de caractères s'écrivent entre apostrophes simples.

VIII- Catégoriser les commandes selon leur montant

3. Affichez d'abord la table des commandes pour repérer les colonnes utiles.

```
SELECT *
FROM sales.salesorderheader;
```

4. Limitez la sélection aux identifiants de commandes et aux montants totaldue.

```
SELECT salesorderid, totaldue
FROM sales.salesorderheader;
```

totaldue représente le montant total dû pour la commande. Les catégories du texte sont High lorsque ce montant dépasse 1000, Medium lorsqu'il se situe entre 500 et 1000 inclus, et Low dans les autres cas. Ce montant ne prouve pas à lui seul que le client a déjà payé.

5. Ajoutez le premier test, puis complétez les catégories avec BETWEEN et ELSE.

```
SELECT salesorderid, totaldue,
       CASE
           WHEN totaldue > 1000 THEN 'High'
           WHEN totaldue BETWEEN 500 AND 1000 THEN 'Medium'
           ELSE 'Low'
       END AS category
FROM sales.salesorderheader;
```

6. Vérifiez quelques lignes de chaque catégorie disponible. Contrôlez spécialement les montants proches de 500 et 1000.

Les montants exactement égaux à 500 et à 1000 appartiennent à Medium ; un montant strictement inférieur à 500 appartient à Low. Pour un montant connu, les trois cas couvrent tous les montants. Si totaldue était NULL, aucun WHEN ne serait vrai et ELSE produirait également Low. Ce cas doit être distingué d'un montant connu inférieur à 500.

7. Exécutez ces tests simples pour vérifier les bornes sans dépendre de leur présence dans la table.

```
SELECT CASE
           WHEN 500 > 1000 THEN 'High'
           WHEN 500 BETWEEN 500 AND 1000 THEN 'Medium'
           ELSE 'Low'
       END AS test_500;
```

Recommencez en remplaçant les deux occurrences de 500 testées à gauche des comparaisons par 1000, puis 1001 et 499. Gardez les bornes 500 et 1000 du BETWEEN inchangées. Les résultats attendus sont Medium, High et Low pour ces trois essais.

8. Relisez le code : utilisez WHEN à l'intérieur de CASE, THEN avant chaque texte, ELSE pour le dernier cas et END avant AS category. Vérifiez la virgule après totaldue et le point-virgule final.

IX- Afficher une couleur de remplacement

9. Affichez les couleurs sans traitement afin d'observer les absences.

```
SELECT productid, color
FROM production.product;
```

10. Appliquez COALESCE avec le texte No color et l'alias product_color.

```
SELECT productid,
       COALESCE(color, 'No color') AS product_color
FROM production.product;
```

11. Affichez la couleur initiale à côté du résultat pour vérifier qu'une couleur renseignée est conservée et qu'une absence est remplacée dans l'affichage.

```
SELECT productid, color,
       COALESCE(color, 'No color') AS product_color
FROM production.product;
```

12. Réécrivez ce traitement avec CASE, comme dans la section II, puis comparez les valeurs. Expliquez pourquoi cette opération ne met pas à jour la table.

X- Calculer le ratio entre prix et coût

13. Affichez les noms, prix catalogue et coûts standard, puis ajoutez la division protégée.

```
SELECT name, listprice, standardcost,
       listprice / NULLIF(standardcost, 0) AS markup
FROM production.product;
```

14. Repérez un coût nul et un coût positif si ces situations existent. Vérifiez le résultat de NULLIF avant d'interpréter le ratio. Une valeur standardcost égale à zéro devient NULL dans le calcul ; elle n'est pas modifiée dans la table.

XI- Comparer une date d'embauche à une date de référence

15. Affichez la table des employés, puis les colonnes businessentityid et hiredate.

```
SELECT businessentityid, hiredate
FROM humanresources.employee;
```

16. Reproduisez le traitement NULLIF avec une date écrite explicitement. La date de référence ci-dessous correspond au jour de préparation du TP ; remplacez-la par celle de votre séance si nécessaire.

```
SELECT businessentityid, hiredate,
       NULLIF(hiredate, '2026-11-08') AS hire_date_result
FROM humanresources.employee;
```

Si hiredate est une date égale à la référence, hire_date_result vaut NULL ; sinon, la date initiale est conservée. Il peut n'exister aucune correspondance dans une sauvegarde historique d'AdventureWorks : cela ne rend pas la requête incorrecte. Vérifiez le type de hiredate ; si la colonne contient aussi une heure, l'égalité porte sur la valeur temporelle complète.

Le texte annonce d'abord des noms d'employés et un libellé « New Hire », puis sa démonstration sélectionne un identifiant et produit NULL. Ce TP suit la démonstration : businessentityid et NULLIF. NULLIF ne produit pas ce libellé textuel. La récupération des noms et les fonctions de date ne sont pas introduites ici.

17. Pour observer l'égalité avec les données existantes, relevez une date hiredate et utilisez-la comme deuxième argument de NULLIF. Comparez ensuite la date d'origine et le résultat.

PARTIE FINALE TRAVAIL À RENDRE

Exercice 1- Comprendre les expressions

1. Expliquez le rôle de CASE, WHEN, THEN, ELSE et END.

2. Que retourne CASE si aucun WHEN n'est vrai et si ELSE est absent ?

3. Donnez le résultat de COALESCE(NULL, 'Black', 'No color'), NULLIF(5, 5) et NULLIF(5, 0).

4. Pourquoi le ratio protégé par NULLIF peut-il être NULL au lieu de zéro ?

5. À partir de la figure 1, indiquez les résultats associés aux codes 3, 6 et 8. Expliquez le dernier cas.

Exercice 2- Catégoriser les commandes

Q1. Afficher salesorderid, totaldue et la catégorie High, Medium ou Low avec les seuils du texte. – 31465 rows

Q2. Justifier les catégories obtenues pour 499, 500, 1000 et 1001.

Q3. Expliquer l'effet de ELSE sur un montant NULL éventuel et l'effet de l'absence de ELSE pour une condition non satisfaite.

Exercice 3- Traiter les couleurs et le ratio

Q4. Afficher productid, color et product_color en remplaçant les couleurs NULL par No color avec COALESCE. – 504 rows

Q5. Réécrire Q4 avec CASE et comparer les résultats. – 504 rows

Q6. Afficher name, listprice, standardcost et le ratio markup avec NULLIF pour éviter une division par zéro. Expliquer les cas coût nul et coût positif. - 504 rows

Contrôle avec les captures : expliquez les résultats de Gloves et Pump dans la figure 2, puis ceux de A, B et E dans la figure 3.

Exercice 4- Appliquer NULLIF aux dates

Q7. Afficher businessentityid, hiredate et une date calculée qui devient NULL lorsque hiredate est égale à une date de référence écrite dans la requête. – 290 rows

Q8. Choisir une date réellement observée, exécuter le traitement puis expliquer ce qui est conservé et ce qui devient NULL. Aucun nom d'employé ni fonction supplémentaire n'est demandé. – 38 rows

Exercice 5- Corriger les erreurs de la démonstration

```
-- A : categoriser les commandes
SELECT salesorderid, totaldue
       CASE
           WHERE totaldue > 1000 THEN 'High'
           ELSE 'Low'
       AS category
FROM sales.salesorderheader;

-- B : afficher No color en cas de couleur absente
SELECT productid, COALESCE('No color', color) AS product_color
FROM production.product;

-- C : proteger le denominateur nul
SELECT name, listprice / NULLIF(0, standardcost) AS markup
FROM production.product;
```

Corrigez chaque requête et expliquez les défauts : séparateur de colonnes et mots-clés pour A, ordre des arguments pour B et C. Dans A, le diagnostic porte sur la structure minimale du CASE à deux résultats ; les trois catégories restent demandées dans Q1.

Remise

Remettez TP10_Nom_Prenom.sql commenté et un court compte rendu avec les réponses, les corrections et les captures de Q1, Q4 et Q6.